UPSKILL CAMPUS • THE IoT ACADEMY • UCT

INDUSTRIAL INTERNSHIP REPORT

PREDICTION OF AGRICULTURE CROP PRODUCTION IN INDIA

Machine Learning / Data Science — Agriculture

Prepared by: Pochampally Akshara

Project 4 | Final Submission

College / Department: [Add college name and logo]

Executive Summary

This report documents the internship project “Prediction of Agriculture Crop Production in India,” in the Machine Learning / Data Science domain. The supplied weekly reports describe Indian agricultural crop-production data for 2001–2014, reported as sourced from data.gov.in.

The work covered dataset understanding, cleaning, exploratory data analysis, feature engineering, baseline regression models, model tuning, evaluation and interpretation. The reports describe Linear Regression, Decision Tree, Random Forest and XGBoost experiments. They state that a tuned ensemble achieved the lowest RMSE and highest R^2 among models tried, with State, Crop and Season identified as dominant predictors.

The original CSV, trained model and numerical evaluation table were not included with the supplied reports. This report therefore does not fabricate scores. Run the included code with the approved dataset to generate actual metrics.

Table of Contents

1. Preface

2. Introduction

3. Problem Statement

4. Existing and Proposed Solution

5. Proposed Design / Model

6. Performance Test

7. My Learnings

8. Future Work Scope

1. Preface

The internship provided an opportunity to apply Python and machine-learning concepts to an agricultural data problem. Work progressed from dataset acquisition and understanding to cleaning, visualization, feature engineering, regression modelling and interpretation. Challenges included mixed data types, missing values, outliers, measurement units, skewed production values and data leakage risk.

Acknowledgement: I thank upskill Campus, The IoT Academy, UniConverge Technologies (UCT), and the mentors and peers who supported the internship. Add individual names if required.

2. Introduction

2.1 About UniConverge Technologies Pvt Ltd

The supplied sample report describes UCT as a company established in 2013 working in digital transformation and industrial solutions, using technologies such as IoT, cybersecurity, cloud computing, machine learning, communication technologies and software development. It also describes UCT Insight and Factory Watch. This agriculture project does not claim integration with those platforms.

2.2 About upskill Campus and The IoT Academy

According to the sample report, upskill Campus, The IoT Academy and UCT facilitated the internship process. The sample describes The IoT Academy as UCT’s EdTech division.

2.3 Internship Objectives

• Gain practical industry-oriented experience.

• Apply Python and data-science methods to agricultural data.

• Improve cleaning, visualization, feature engineering and model evaluation skills.

• Build a reproducible workflow and document it on GitHub.

• Improve technical communication and problem-solving.

2.4 References

• data.gov.in — dataset source identified in the weekly reports; add exact URL and access date.

• Pandas: https://pandas.pydata.org/docs/

• scikit-learn: https://scikit-learn.org/stable/

• Matplotlib: https://matplotlib.org/stable/

• XGBoost: https://xgboost.readthedocs.io/

2.5 Glossary

Term	Meaning
EDA	Exploratory Data Analysis
MAE	Mean Absolute Error
RMSE	Root Mean Squared Error
R^2	Coefficient of determination
Pipeline	Preprocessing and model workflow
Data leakage	Unintended use of information during model fitting

3. Problem Statement

Agricultural production records can contain missing values, inconsistent crop/state names, mixed data types, skewed production quantities and different measurement units. The project investigates a repeatable workflow to clean historical records, explore patterns by crop, state, season and year where available, and estimate production from available attributes.

Objectives

• Audit missing values, duplicates, data types and category consistency.

• Clean and standardize records while documenting decisions.

• Explore production patterns by state, crop, season and year.

• Train regression models and compare MAE, RMSE and R^2.

• Interpret important predictors and save a reusable pipeline.

The weekly reports describe data for 2001–2014 and fields such as Crop, Variety, State, Quantity, Production, Season, Unit, Cost and Recommended Zone. The original CSV was not attached, so the exact schema and units must be confirmed.

4. Existing and Proposed Solution

4.1 Existing Approach and Limitations

Manual summaries and spreadsheets can describe historical production but often require repeated cleaning and may not provide a reusable prediction workflow. Linear regression can struggle with skewed targets and complex relationships. Comparisons can be misleading if measurement units differ.

4.2 Proposed Solution

The proposed workflow uses repeatable cleaning, exploratory analysis, preprocessing pipelines and supervised regression. Weekly reports document Linear Regression, Decision Tree, Random Forest and XGBoost experiments, target log transformation, tuning, cross-validation and feature-importance analysis.

• Fit preprocessing within the training pipeline to reduce leakage.

• Document unit handling and data-quality checks.

• Compare models using held-out data and consistent metrics.

• Save the fitted model for repeatable predictions.

4.3 Submission Links

GitHub repository: [Paste your actual repository URL after creating and uploading it].

5. Proposed Design / Model

The solution starts with a CSV dataset, validates and cleans records, performs exploratory analysis and feature engineering, fits a preprocessing-and-regression pipeline, evaluates held-out predictions and saves the selected model.

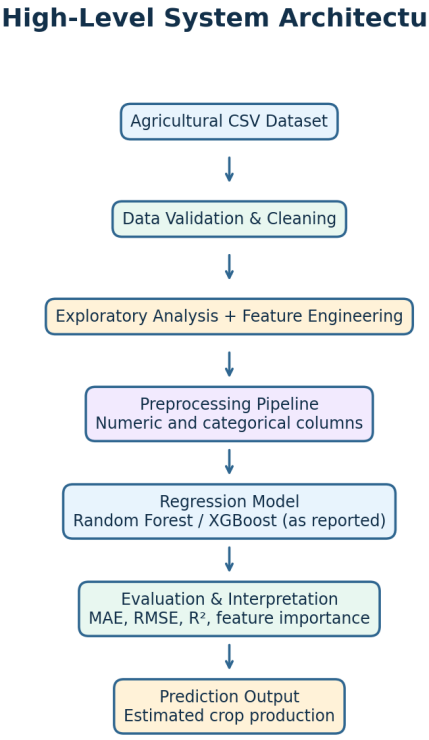


Figure 1. High-level architecture of the crop production prediction system.

5.1 High-Level Diagram

The architecture separates raw data, preparation, feature transformation, model training, evaluation and prediction output.

5.2 Low-Level Workflow

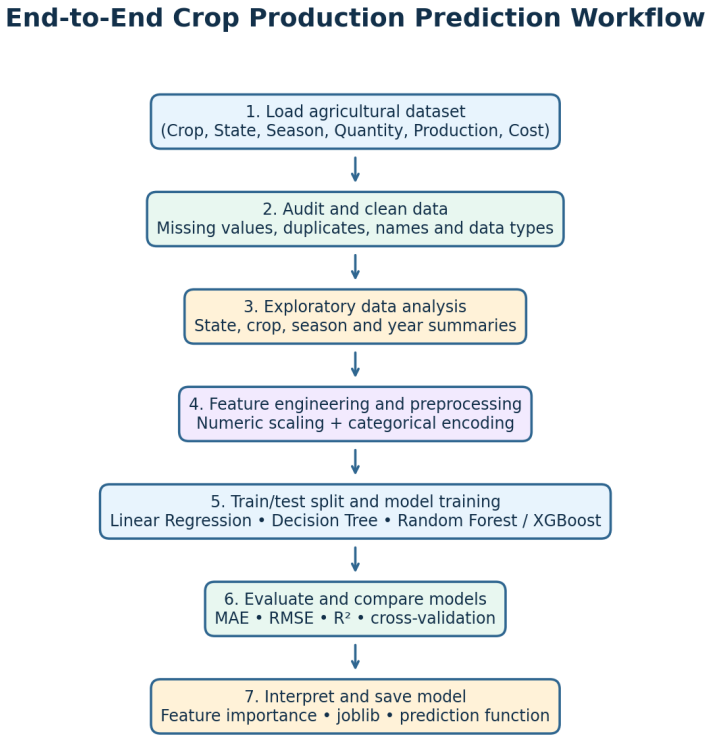


Figure 2. End-to-end workflow based on stages documented in weekly reports.

5.3 Interfaces and Outputs

• Input: CSV containing Production and available agricultural predictors.

• Preprocessing: duplicate removal, numeric parsing, imputation, encoding and scaling.

• Training: regression algorithms fit on a training split.

• Evaluation: MAE, RMSE and R^2 on held-out records.

• Outputs: metrics CSV, saved pipeline and prediction values.

The included Python implementation is a runnable baseline; it does not reproduce every experiment described in the weekly reports.

6. Performance Test

Performance testing checks pipeline execution, reproducibility and generalization to unseen records. The supplied reports describe metrics but do not include exact scores.

6.1 Test Plan / Test Cases

Test	Procedure	Expected check
Load data	Read approved CSV	Target Production exists
Clean data	Inspect duplicates, missing target, numeric fields	Cleaning decisions are documented
Leakage	Fit transforms inside training pipeline	Test set not used to fit preprocessing
Train	Fit regression baselines	Training completes
Evaluate	Calculate MAE, RMSE, R^2	Metrics CSV is generated

Test	Procedure	Expected check
Predict	Load model and new inputs	Prediction output is generated

6.2 Test Procedure

• Install dependencies from requirements.txt.

• Place the authorized dataset at data/crop_production.csv.

• Run the training script and inspect metrics.

• Review generated metrics and saved model.

• Run prediction with the same predictor columns.

• Record dataset version, software versions, command and results.

6.3 Performance Outcome

The Week 4 agriculture report states that Random Forest and XGBoost were tuned using GridSearchCV / RandomizedSearchCV; the tuned ensemble gave the lowest RMSE and highest R^2 among models tried. It reports that log transformation improved errors on skewed high-production records and that State, Crop and Season were dominant predictors.

Exact MAE, RMSE, R^2, split details and trained artifact were not supplied. No numerical performance claims are made here. Run the code on the original dataset before adding measured scores.

Model Development and Evaluation Plan

Model	Purpose	Evaluation status in supplied reports
Linear Regression	Baseline regression	Trained; lower performance noted on skewed target
Decision Tree Regressor	Non-linear baseline	Trained; numerical metrics not supplied
Random Forest Regressor	Ensemble model	Tuned; reported as best among models tried
XGBoost Regressor	Boosted ensemble	Tuned; reported as competitive / part of best tuned ense

Figure 3. Qualitative summary of model development reported in the weekly reports; not a numerical performance chart.

7. My Learnings

• Used Pandas and NumPy to inspect real-world tabular data.

• Practised missing-value handling, duplicate removal and category standardization.

• Used Matplotlib and Seaborn for exploratory visualization.

• Applied groupby, pivot_table and merge for summaries.

• Built scikit-learn preprocessing and regression pipelines.

• Learned MAE, RMSE, R^2, train/test splitting and cross-validation.

• Recognized data leakage risks and the need for unit consistency.

• Explored target transformation, model tuning, feature importance and joblib persistence.

8. Future Work Scope

• Add reliable weather, rainfall, soil, irrigation, pest and market-price data.

• Use time-based validation and test generalization across regions.

• Normalize units carefully and document feature definitions.

• Compare Random Forest and XGBoost under consistent evaluation.

• Add prediction uncertainty and explanations.

• Build a dashboard and automated data-quality checks.

• Add measured metrics, actual output graphs and screenshots from the dataset.

Appendix A — Running the Code

Install Python 3.10+, create a virtual environment, run `pip install -r requirements.txt`, add the authorized CSV to data/crop_production.csv, then run `python src/train_model.py --data data/crop_production.csv`. Use src/predict.py with an input CSV containing the same predictor columns.

Appendix B — Submission Checklist

• Add college logo and college name.

• Add exact dataset URL and access date.

• Run code on the actual dataset and include real metrics.

• Create GitHub repository and replace placeholder URL.

• Upload report, README, source code and requirements.txt.

• Keep code submission package under 10 MB.

• Upload the required social-media screenshot to the Google Form.